RLVR_Yash_First

by Yash Ingle

Submission date: 24-Apr-2026 03:32PM (UTC+0530)
Submission ID: 2942315700
File name: RLVR_Research_paper.pdf (602.06K)
Word count: 7306
Character count: 37480

Adaptive Negative Reinforcement for LLM Reasoning: Dynamically Balancing Correction and Diversity in RLVR

Yash Ingle Jaival Chauhan Ankit Yadav Sudhakar Mishra
Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat, India
{U23AI062, U23AI035, U23AI039, sudhakar.mishra}@coed.svnit.ac.in

April 24, 2026

Abstract

Reinforcement learning with verifiable rewards (RLVR) has become an good approach for improving reasoning in language models (LMs). An insight from earlier work is that Negative Sample Reinforcement (NSR)—which focuses on penalizing incorrect responses without explicitly rewarding correct ones—can match or even better standard reinforcement learning methods such as PPO and GRPO across the full Pass@k spectrum. Despite this, most existing NSR approaches depends on a *fixed* balance between positive and negative updates throughout training and treat all incorrect responses in the same way. This ignores two important aspects of the learning: the fact that the model's error patterns increase over time, and that not all mistakes are equally useful.

To address these limitations, we propose two independent and complementary extensions. First, **Adaptive Negative Sample Reinforcement (A-NSR)** introduces time-dependent scheduling functions that adjust the strength of positive and negative updates across training. In the early training time, it focuses on fixing mistakes more strongly, and later it shifts to more controlled updates that help maintain diversity. Second, **Confidence-Weighted Negative Reinforcement (CW-NSR)** assigns sample-specific penalty weights based on the model's normalized sequence likelihood, so that highly confident mistakes are penalized more strongly than uncertain ones. We provide a gradient-level analysis showing how these mechanisms control token-level updates while keeping the key benefits of standard NSR, such as prior-guided probability redistribution and implicit regularization against overfitting. Experiments on MATH, AIME 2025, and AMC23 using Qwen2.5-Math-7B and Qwen3-4B mdemonstrate that both A-NSR and CW-NSR, individually as well as together, consistently improve Pass@k performance across a wide range (k from 1 to 256), achieving a more good trade-off between accuracy and diversity compared to static Weighted-REINFORCE, GRPO, and PPO.

1 Introduction

Language models (LMs) have recently shown strong performance on a wide range of complex reasoning tasks, including mathematical problem solving [Hendrycks et al.(2021)], [Cobbe et al.(2021)], code generation [Jimenez et al.(2024)], and scientific reasoning [Rein et al.(2024)]. A major reason behind this progress is *Reinforcement Learning with Verifiable Rewards* (RLVR) [Lambert et al.(2024)], [Guo et al.(2025)], [Kimi Team et al.(2025)], where models are trained using simple binary feedback (+1 for correct, −1 for incorrect) verified through deterministic checkers. This setup avoids many of the issues linked with learned reward models, such as reward hacking [Skalse et al.(2022)], and keeps the training signal straightforward.

A good observation from Zhu et al. [Zhu et al.(2025)] is that the *negative sample reinforcement* (NSR) component of RLVR—penalizing incorrect outputs—can be surprisingly effective on its own. In many cases, it matches or even does better than methods like PPO [Schulman et al.(2017)] and GRPO [Shao et al.(2024)] across the full Pass@k range. Their analysis shows that NSR works by disappointing incorrect reasoning paths while redistributing probability mass toward more good alternatives, guided by the model's prior. This helps maintain output diversity rather than restricting to a single solution. Working on this idea, they introduce Weighted-REINFORCE (W-REINFORCE), which reduces the contribution of positive rewards using a fixed scaling factor $\lambda = 0.1$.

Although it has effectiveness, but the fixed nature of W-REINFORCE still has some areas for improvement in two important ways:

1. **Temporal dimension.** The model's error patterns change quite a bit during training. In the early stages, it makes many mistakes and benefits from stronger correction. Later on, once accuracy improves, pushing too hard can start to reduce the diversity that makes NSR useful in the first place. A fixed λ does not really adjust to this shift.
2. **Sample dimension.** Not every wrong answer deserves the same treatment. A mistake made with high confidence usually shows to a more serious failure mode and should be corrected more strongly, while a low-confidence mistake is often just an uncertain guess and may only need a light penalty. Treating both in the same way blurs this difference.

In this paper, we address both limitations with two independent and complementary proposals:

Adaptive Negative Sample Reinforcement (A-NSR). We introduce time-dependent scheduling functions $\lambda(t)$ and $\beta(t)$ to adjust the strength of PSR and NSR as training progresses. In particular, we use an exponential decay for the NSR weight and a linear increase for the PSR weight. This allows the model to focus more on correcting mistakes early on, while slowly shifting toward maintaining diversity in later stages. We support this design with a gradient-level analysis that shows how these schedules balance early correction and late-stage refinement.

Confidence-Weighted Negative Reinforcement (CW-NSR). We define a per-sample *hardness score* using the model's normalized sequence likelihood, calculated as the geometric mean of token probabilities. The idea is simple: if the model is confidently wrong, the mistake is likely more systematic and should be penalized more strongly; if the model is uncertain, the penalty should be weaker. We show that this weighting scheme holds the core behavior of NSR—redistributing probability mass based on the model's prior—while focusing updates on the most problematic errors.

Taken together, A-NSR determines *when* stronger negative reinforcement should be applied, while CW-NSR determines *which* incorrect samples should receive it. The two operate at different levels and can be combined naturally within the same training objective.

Contributions.

- We propose A-NSR, a time-dependent scheduling framework for RLVR that adapts the PSR/NSR balance to training stage (§3.1).
- We propose CW-NSR, a confidence-based per-sample weighting mechanism that focuses negative reinforcement on high-confidence errors (§3.2).
- We provide token-level gradient analysis showing that both mechanisms preserve NSR's desirable properties while making it more selective (§4).
- We demonstrate consistent improvements over W-REINFORCE, GRPO, and PPO on MATH, AIME 2025, and AMC23 across the full Pass@k spectrum (§5).

2 Background and Preliminaries

2.1 Reinforcement Learning with Verifiable Rewards

Given a language model with parameters θ, a prompt distribution $\mathcal{D}$, and a deterministic verification function r, RLVR optimizes:

$$\mathcal{L}_{\text{RLVR}}(\theta) = -\mathbb{E}_{\boldsymbol{x}\sim\mathcal{D},\ \boldsymbol{y}\sim\pi_\theta(\cdot|\boldsymbol{x})}\big[r(\boldsymbol{x},\boldsymbol{y})\big], \qquad r(\boldsymbol{x},\boldsymbol{y}) \in \{-1,+1\}. \tag{1}$$

2.2 PSR–NSR Decomposition

Following Zhu et al. Zhu et al.(2025), the objective decomposes into:

$$\mathcal{L}_{\text{RLVR}}(\theta) = \mathcal{L}_{\text{PSR}}(\theta) + \mathcal{L}_{\text{NSR}}(\theta), \tag{2}$$

where

$$\mathcal{L}_{\text{PSR}}(\theta) = -\mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Bigg[\sum_{\boldsymbol{y}:\, r(\boldsymbol{x},\boldsymbol{y})=1} \pi_\theta(\boldsymbol{y}|\boldsymbol{x})\Bigg], \tag{3}$$

$$\mathcal{L}_{\text{NSR}}(\theta) = -\mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\Bigg[\sum_{\boldsymbol{y}:\, r(\boldsymbol{x},\boldsymbol{y})=-1} -\pi_\theta(\boldsymbol{y}|\boldsymbol{x})\Bigg]. \tag{4}$$

Weighted-REINFORCE Zhu et al.(2025). To balance accuracy and diversity, the W-REINFORCE objective scales PSR by a fixed constant λ:

$$\mathcal{L}_{\text{W-REINFORCE}}(\theta) = \lambda\, \mathcal{L}_{\text{PSR}}(\theta) + \mathcal{L}_{\text{NSR}}(\theta), \qquad \lambda = 0.1. \tag{5}$$

2.3 Token-Level Gradient Dynamics

For a training instance $(\boldsymbol{x}, \boldsymbol{y})$ with reward $R = r(\boldsymbol{x}, \boldsymbol{y})$, the per-token loss is:

$$\mathcal{L}(\theta) = -R \cdot \frac{1}{T}\sum_{t=1}^{T} \pi_\theta(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t}). \tag{6}$$

Let $\pi_v = \pi_\theta(v|\boldsymbol{x}, \boldsymbol{y}_{<t})$. The gradient descent direction with respect to the logit z_v of token v at step t is Zhu et al.(2025):

For PSR ($R = +1$):

$$-\frac{\partial \mathcal{L}_{\text{PSR}}}{\partial z_v} \propto \begin{cases} \pi_v(1-\pi_v) & \text{if } v = y_t \quad \text{(sampled token)}, \\ -\pi_{y_t}\pi_v & \text{if } v \neq y_t \quad \text{(unsampled token)}. \end{cases} \tag{7}$$

For NSR ($R = -1$):

$$-\frac{\partial \mathcal{L}_{\text{NSR}}}{\partial z_v} \propto \begin{cases} -\pi_v(1-\pi_v) & \text{if } v = y_t, \\ \pi_{y_t}\pi_v & \text{if } v \neq y_t. \end{cases} \tag{8}$$

The key properties of NSR highlighted by Zhu et al.(2025) can be summarized as follows: (i) *preserving high-confidence priors*: penalties on confident tokens scale with $(1 - \pi_{y_t})$, which helps avoid erasing useful pretrained knowledge; (ii) *prior-guided redistribution*: the logits of unsampled tokens are increased in proportion to π_v, allowing probability mass to shift toward possible alternatives; and (iii) *implicit regularization*: the updates naturally decrease once the model stops producing incorrect responses.

3 Method

We now describe our two extensions in more detail. Section 3.1 presents Adaptive NSR, which adjusts the strength of reinforcement over the course of training. Section 3.2 introduces Confidence-Weighted NSR, which varies the penalty across individual samples. Finally, Section 3.3 discusses how the two can be combined in a straightforward way.

3.1 Adaptive Negative Sample Reinforcement (A-NSR)

Motivation. The model's error distribution changes quite noticeably during RLVR training. As observed by Zhu et al. Zhu et al.(2025) (Figure 5c–d), the proportion of correct samples increases from approximately ~ 0.5 to above 0.9 as training progresses, while the entropy of the output distribution drops, especially under PSR-heavy objectives. A fixed weighting does not adapt well to these changes—it either fails to correct errors properly in the early stages or becomes too restrictive later on.

Objective. We replace the fixed λ with time-dependent scheduling functions:

$$\mathcal{L}_{\text{A-NSR}}(\theta; t) = \lambda(t)\,\mathcal{L}_{\text{PSR}}(\theta) + \beta(t)\,\mathcal{L}_{\text{NSR}}(\theta), \tag{9}$$

where t is the current training step and $\lambda(t), \beta(t) : \mathbb{N} \to \mathbb{R}^+$ are scheduling functions satisfying $\lambda(t), \beta(t) > 0$ for all t.

Gradient interpretation. The standard policy gradient takes the form:

$$\nabla_\theta \mathcal{L}_{\text{RLVR}} = \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}}\big[r(\boldsymbol{x},\boldsymbol{y}) \nabla_\theta \log \pi_\theta(\boldsymbol{y}|\boldsymbol{x})\big]. \tag{10}$$

Under adaptive weighting, the effective gradient becomes:

$$\nabla_\theta \mathcal{L}_{\text{A-NSR}} = \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}}\big[w(t,r)\, r(\boldsymbol{x},\boldsymbol{y})\, \nabla_\theta \log \pi_\theta(\boldsymbol{y}|\boldsymbol{x})\big], \tag{11}$$

where

$$w(t,r) = \begin{cases} \lambda(t), & r(\boldsymbol{x},\boldsymbol{y}) = +1, \\ \beta(t), & r(\boldsymbol{x},\boldsymbol{y}) = -1. \end{cases} \tag{12}$$

This shows that A-NSR reweights the magnitude—but not the direction—of PSR and NSR gradients over time.

Scheduling functions. We consider three principled schedules:

Schedule 1: Exponential decay for NSR, with a linear increase for PSR.

$$\beta(t) = \beta_{\min} + (\beta_{\max} - \beta_{\min})\, e^{-\kappa t}, \tag{13}$$

$$\lambda(t) = \lambda_{\min} + (\lambda_{\max} - \lambda_{\min})\, \frac{t}{T_{\text{total}}}. \tag{14}$$

This implements a smooth curriculum: strong correction early, balanced updates in the middle, and softer negative reinforcement later.

Schedule 2: Cosine annealing for NSR.

$$\beta(t) = \beta_{\min} + \frac{1}{2}(\beta_{\max} - \beta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T_{\text{total}}}\right)\right). \tag{15}$$

This provides a gradual, smooth transition without the sharp early decay of the exponential schedule.

Schedule 3: Performance-driven adaptive weight.

$$\beta(t) = \beta_{\min} + (\beta_{\max} - \beta_{\min}) \cdot (1 - \hat{p}_{\text{correct}}(t)), \tag{16}$$

where $\hat{p}_{\text{correct}}(t)$ is the empirical correct sample ratio in the current training batch. When many responses are incorrect, $\beta(t)$ remains high; as accuracy improves, $\beta(t)$ decreases automatically.

Proposition 1 (Convergence of effective weight)**.** *Under Schedule 1 (Eqs. 13, 14), the ratio of NSR to PSR gradient magnitude converges:*

$$\frac{\beta(t)}{\lambda(t)} \xrightarrow{t \to T_{\text{total}}} \frac{\beta_{\min}}{\lambda_{\max}}.$$

Thus, in the later stages of training, the objective behaves like a fixed W-REINFORCE with an effective coefficient $\lambda_{\text{eff}} = \lambda_{\max}/\beta_{\min}$, while the early stages are dominated by NSR, with a ratio of $\beta_{\max}/\lambda_{\min}$.

Relationship to curriculum learning. A-NSR can be viewed as a form of curriculum learning Bengio et al.(2009) applied to the reward signal rather than the data. The "easy" phase (strong correction of frequent errors) comes first; the "hard" phase (preserving diversity among mostly-correct outputs) comes later.

3.2 Confidence-Weighted Negative Reinforcement (CW-NSR)

Motivation. Even within a single training batch, not all incorrect responses are equally severe. A wrong answer given with high confidence often points to a systematic bias and should be corrected more strongly. In contrast, a low-confidence mistake may just be an exploratory attempt, and penalizing it too heavily could limit useful exploration. Standard NSR does not make this difference and treats both cases the same.

Confidence from sequence likelihood. For a response $\boldsymbol{y} = (y_1, \ldots, y_T)$, the model assigns probabilities in an autoregressive manner:

$$\pi_\theta(\boldsymbol{y}|\boldsymbol{x}) = \prod_{t=1}^{T} \pi_\theta(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t}). \tag{17}$$

To remove length dependence, we define the *confidence score* as the geometric mean token probability:

$$\mathrm{Conf}(\boldsymbol{y}) = \exp\left(\frac{1}{T}\sum_{t=1}^{T} \log \pi_\theta(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})\right) = (\pi_\theta(\boldsymbol{y}|\boldsymbol{x}))^{1/T}. \tag{18}$$

Remark 1. $\mathrm{Conf}(\boldsymbol{y}) \in (0, 1]$ *and is invariant to response length* T*. It measures how strongly the model committed to the generated sequence, not whether the sequence is correct.*

Hardness weighting function. We define a per-sample hardness weight for incorrect responses:

$$w(\boldsymbol{y}) = \max\big(\epsilon,\ \mathrm{Conf}(\boldsymbol{y})^\alpha\big), \tag{19}$$

where $\alpha > 0$ controls sensitivity to confidence and $\epsilon > 0$ is a floor ensuring every incorrect sample receives a minimum penalty.

Objective. The CW-NSR objective replaces the uniform negative penalty with hardness-weighted penalties:

$$\mathcal{L}_{\text{CW-NSR}}(\theta) = \lambda \mathcal{L}_{\text{PSR}}(\theta) + \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\left[\sum_{\boldsymbol{y}:\, r(\boldsymbol{x},\boldsymbol{y})=-1} w(\boldsymbol{y}) \cdot \big(-\pi_\theta(\boldsymbol{y}|\boldsymbol{x})\big)\right]. \tag{20}$$

Gradient analysis. The token-level gradient of the CW-NSR loss for an incorrect sample $(\boldsymbol{x}, \boldsymbol{y})$ with hardness $w(\boldsymbol{y})$ is:

$$-\frac{\partial \mathcal{L}_{\text{CW-NSR}}}{\partial z_v} \propto w(\boldsymbol{y}) \cdot \begin{cases} -\pi_v(1-\pi_v) & \text{if } v = y_t, \\ \pi_{y_t}\pi_v & \text{if } v \neq y_t. \end{cases} \tag{21}$$

Comparing with Eq. 8, CW-NSR preserves the *direction* of the NSR gradient but scales its *magnitude* by $w(\boldsymbol{y})$. This means:

- **Prior-guided redistribution is preserved**: unsampled tokens are still improved proportionally to their current probability π_v.
- **High-confidence priors are still protected**: the $(1 - \pi_v)$ damping factor remains.
- **Selective penalization**: the *overall* gradient magnitude becomes larger for confident errors and smaller for uncertain ones.

Proposition 2 (Gradient magnitude ordering)**.** *Let* $\boldsymbol{y}^{(1)}$ *and* $\boldsymbol{y}^{(2)}$ *be two incorrect responses with* $\mathrm{Conf}(\boldsymbol{y}^{(1)}) > \mathrm{Conf}(\boldsymbol{y}^{(2)})$*. Then for any token* v *and any* $\alpha > 0$*:*

$$\left\| \frac{\partial \mathcal{L}_{\text{CW-NSR}}}{\partial z_v}\Big|_{\boldsymbol{y}^{(1)}} \right\| \geq \left\| \frac{\partial \mathcal{L}_{\text{CW-NSR}}}{\partial z_v}\Big|_{\boldsymbol{y}^{(2)}} \right\|,$$

i.e., more confident errors always produce larger gradient updates.

Proof. Since $\alpha > 0$ and $\mathrm{Conf}(\boldsymbol{y}^{(1)}) > \mathrm{Conf}(\boldsymbol{y}^{(2)})$, we have $w(\boldsymbol{y}^{(1)}) = \mathrm{Conf}(\boldsymbol{y}^{(1)})^\alpha > \mathrm{Conf}(\boldsymbol{y}^{(2)})^\alpha = w(\boldsymbol{y}^{(2)})$ (assuming both exceed ϵ). The gradient magnitude is proportional to $w(\boldsymbol{y})$, and the remaining factors depend only on the current policy π_θ at step t, not on the sample. The result follows. □

Algorithm 1 Combined A-NSR + CW-NSR Training

Require: Policy π_θ, prompt dataset $\mathcal{D}$, schedules $\lambda(\cdot)$, $\beta(\cdot)$, confidence exponent α, floor ϵ

for $t = 1, 2, \ldots, T_{\text{total}}$ **do**
  Sample batch of prompts $\{\boldsymbol{x}_i\}_{i=1}^{B}$ from $\mathcal{D}$
  for each prompt $\boldsymbol{x}_i$ **do**
    Generate G rollouts $\{\boldsymbol{y}_i^{(g)}\}_{g=1}^{G}$ from $\pi_\theta(\cdot|\boldsymbol{x}_i)$
    Compute rewards $r_i^{(g)} = r(\boldsymbol{x}_i, \boldsymbol{y}_i^{(g)}) \in \{-1, +1\}$
    for each rollout $\boldsymbol{y}_i^{(g)}$ with $r_i^{(g)} = -1$ **do**
      Compute $\text{Conf}(\boldsymbol{y}_i^{(g)})$ via Eq. 18
      Compute hardness $w(\boldsymbol{y}_i^{(g)}) = \max(\epsilon, \text{Conf}(\boldsymbol{y}_i^{(g)})^\alpha)$
    end for
  end for
  Compute adaptive weights $\lambda(t)$, $\beta(t)$
  Compute loss $\mathcal{L}_{\text{Combined}}(\theta; t)$ via Eq. 22
  Update θ with gradient descent (with clipping)
end for

Connection to hard-example mining. CW-NSR can be seen as a soft form of hard-example mining [Shrivastava et al.(2016)]: confident errors are "hard" negatives that the model needs to forget, while uncertain errors are "easy" negatives that will naturally decrease as training progresses.

3.3 Combined Objective: A-NSR + CW-NSR

The two mechanisms operate at different levels—over training time and across individual samples—and can be combined in a straightforward way:

$$\boxed{\mathcal{L}_{\text{Combined}}(\theta; t) = \lambda(t)\, \mathcal{L}_{\text{PSR}}(\theta) + \beta(t)\, \mathbb{E}_{\boldsymbol{x}}\Big[\sum_{\boldsymbol{y}:\, r=-1} w(\boldsymbol{y}) \cdot \big(-\pi_\theta(\boldsymbol{y}|\boldsymbol{x})\big)\Big].} \tag{22}$$

The effective per-sample, per-timestep weight for an incorrect response is $\beta(t) \cdot w(\boldsymbol{y})$, which depends on both the training stage and the model's confidence in that particular error.

Implementation. Algorithm 1 draws the overall training procedure. The additional computational cost is less: $\text{Conf}(\boldsymbol{y})$ is computed from the same forward pass used to generate rollouts, and the scheduling functions only depend on the current training step.

4 Theoretical Analysis

4.1 Token-Level Dynamics Under Adaptive Weighting

We analyze how A-NSR modifies the token-level gradient dynamics identified by Zhu et al.(2025).

Proposition 3 (Entropy preservation under A-NSR)**.** *Let $H_t = -\sum_{v \in \mathcal{V}} \pi_v \log \pi_v$ be the output entropy at time t. Under A-NSR with decreasing $\beta(t)$, the rate of entropy decrease from NSR updates slows over training:*

$$\left|\frac{dH_t}{dt}\right|_{NSR} \propto \beta(t) \cdot \pi_{y_t}(1 - \pi_{y_t}).$$

Since $\beta(t)$ decreases, the NSR-induced entropy change decreases, helping keep diversity in late training.

Proof. The entropy change from a single NSR gradient step on token $v = y_t$ is:

$$\left.\frac{dH_t}{dt}\right|_{\text{NSR}} = -\sum_{v\in\mathcal{V}} \frac{\partial H}{\partial \pi_v} \cdot \frac{\partial \pi_v}{\partial t} = -\sum_{v} (1 + \log \pi_v) \cdot \beta(t) \cdot \Delta\pi_v,$$

where $\Delta\pi_v$ is the change in π_v from the NSR update. Since the NSR gradient (Eq. 8) is scaled by $\beta(t)$, all $\Delta\pi_v$ terms are proportional to $\beta(t)$, and the magnitude of the entropy change is proportional to $\beta(t)\cdot\pi_{y_t}(1-\pi_{y_t})$. □

4.2 Effective Learning Rate Interpretation

A-NSR can be explained as applying different effective learning rates to positive and negative samples:

$$\eta^{+}_{\text{eff}}(t) = \eta \cdot \lambda(t), \qquad \eta^{-}_{\text{eff}}(t) = \eta \cdot \beta(t), \tag{23}$$

where η is the base learning rate. The ratio

$$\rho(t) = \frac{\eta^{-}_{\text{eff}}(t)}{\eta^{+}_{\text{eff}}(t)} = \frac{\beta(t)}{\lambda(t)} \tag{24}$$

captures the relative emphasis on correction versus reinforcement. Under Schedule 1 (Eqs. 13-14):

$$\rho(t) = \frac{\beta_{\min} + (\beta_{\max} - \beta_{\min})e^{-\kappa t}}{\lambda_{\min} + (\lambda_{\max} - \lambda_{\min})\frac{t}{T_{\text{total}}}}, \tag{25}$$

which is repetitively decreasing under mild conditions, smoothly shifting from NSR-dominated to PSR-inclusive training.

4.3 CW-NSR and the Bias–Variance Trade-off

Proposition 4 (Variance reduction)**.** *Let $\hat{g}_{NSR}$ be the NSR policy-gradient estimator with uniform weighting and $\hat{g}_{CW\text{-}NSR}$ be the CW-NSR estimator. Assuming incorrect samples with higher confidence have higher gradient norms, CW-NSR reduces the variance contribution from low-confidence samples:*

$$\text{Var}(\hat{g}_{CW\text{-}NSR}) \le \text{Var}(\hat{g}_{NSR}) + C \cdot \mathbb{E}[(\text{Conf}(\boldsymbol{y})^{\alpha} - 1)^2],$$

where C depends on the policy. When $\alpha < 1$, the increase in weight stays limited, and the variance does not grow too much.

Remark 2. *While CW-NSR can increase gradient variance (since it puts more weight on confident errors), it also reduces the number of updates spent on low-confidence mistakes that carry little useful signal. In practice, we observe that this leads to better sample efficiency overall.*

4.4 Comparison with Entropy Bonus and Unlikelihood Training

We compare the gradient dynamics of our methods with two related approaches:

Entropy bonus. As shown by Zhu et al.(2025), the entropy gradient is:

$$\frac{\partial H}{\partial z_v} = -\pi_v \left(\log \pi_v - \sum_{v'} \pi_{v'} \log \pi_{v'} \right). \tag{26}$$

Unlike NSR, entropy maximization does not condition on sample correctness and can cut off correct high-confidence tokens. Our methods preserve the conditional structure of NSR while adding selectivity.

Unlikelihood training [Welleck et al.(2020)]. The unlikelihood gradient is:

$$-\frac{\partial \mathcal{L}_{\text{UL}}}{\partial z_v} \propto \begin{cases} -\pi_v & \text{if } v = y_t, \\ \frac{\pi_{y_t}}{1-\pi_{y_t}}\pi_v & \text{if } v \neq y_t. \end{cases} \tag{27}$$

The $\frac{1}{1-\pi_{y_t}}$ factor adds updates for confident tokens, which can waste pretrained knowledge. NSR's $(1-\pi_v)$ damping factor avoids this, and our CW-NSR preserves this protection while adding sample-level selectivity.

4.5 Extension to PPO and GRPO

Our methods compose naturally with clipped policy-gradient objectives. For PPO [Schulman et al.(2017)]:

$$\mathcal{L}_{\text{A-PPO}}(\theta; t) = -\frac{1}{T}\sum_{t'=1}^{T} \min\left(\frac{\pi_\theta}{\pi_{\text{old}}} w(t,r) A_{t'},\ \text{clip}\left(\frac{\pi_\theta}{\pi_{\text{old}}}, 1-\epsilon, 1+\epsilon\right) w(t,r) A_{t'}\right), \tag{28}$$

where $w(t,r)$ includes both the adaptive schedule and (optionally) the confidence weight. Clipping limits the update magnitude but preserves the gradient direction, so our analysis remains qualitatively valid.

5 Experiments

5.1 Experimental Setup

Models. We have evaluated on three base models:

- `Qwen2.5-Math-7B` [Yang et al.(2025b)]: A math-specialized model with strong reasoning priors.
- `Qwen3-4B` [Yang et al.(2025a)] (non-thinking mode): Demonstrates transfer from latent thinking capabilities.
- `Llama-3.1-8B-Instruct` [Dubey et al.(2024)]: An instruction-tuned model with weaker math priors.

Training setup. We use the MATH [Hendrycks et al.(2021)] training set (7,500 problems) and the verl framework [Sheng et al.(2024)]. The prompt batch size is 1,024 with 8 rollouts per prompt. Sampling temperature is 1.0, maximum context length is 4,096 tokens for Qwen2.5-Math-7B and Llama-3.1-8B-Instruct, and 32,768 for Qwen3-4B. We use a mini-batch size of 256, learning rate 1e−6, clip ratio $\epsilon = 0.2$, and entropy bonus coefficient 1e−4. All experiments use 8 NVIDIA H200 GPUs.

A-NSR hyperparameters. For Schedule 1 (Eqs. 13, 14): $\beta_{\max} = 1.5$, $\beta_{\min} = 0.5$, $\kappa = 0.03$, $\lambda_{\min} = 0.05$, $\lambda_{\max} = 0.2$. For Schedule 2 (Eq. 15): same β range. For Schedule 3 (Eq. 16): same β range with batch-level correct ratio.

CW-NSR hyperparameters. Confidence exponent $\alpha = 1.0$, floor $\epsilon = 0.1$.

Baselines. We compare against:

- **Base Model**: No RL training.
- **PPO** [Schulman et al.(2017)]: Standard proximal policy optimization.
- **GRPO** [Shao et al.(2024), Guo et al.(2025)]: Group relative policy optimization.
- **REINFORCE**: Standard REINFORCE with equal PSR and NSR weights ($\lambda = 1$).
- **NSR-only**: Training with only negative samples [Zhu et al.(2025)].
- **W-REINFORCE** [Zhu et al.(2025)]: Fixed $\lambda = 0.1$.

Table 1: Pass@k results on MATH, AIME 2025, and AMC23 with `Qwen2.5-Math-7B`. **Bold** and underlined numbers denote the best and second-best results for each k.

Method	Pass@k								
	1	2	4	8	16	32	64	128	256
MATH									
Base Model	63.2	76.0	83.7	88.4	91.6	93.7	95.2	96.2	96.9
PPO	76.6	82.6	86.7	89.6	91.7	93.4	94.7	95.6	96.3
GRPO	76.3	81.7	85.6	88.4	90.6	92.3	93.6	94.7	95.5
REINFORCE	74.8	79.1	82.4	84.9	86.9	88.5	89.9	91.1	92.0
NSR-only	75.7	82.4	86.9	90.1	92.4	94.1	95.3	96.2	96.9
W-REINFORCE	76.6	82.8	87.1	90.2	92.4	94.1	95.3	96.1	96.7
A-NSR (Ours)	–	–	–	–	–	–	–	–	–
CW-NSR (Ours)	–	–	–	–	–	–	–	–	–
Combined (Ours)	–	–	–	–	–	–	–	–	–
AIME 2025									
Base Model	6.1	9.7	13.8	17.9	22.2	26.5	30.8	36.6	46.7
PPO	8.5	13.2	18.0	22.5	26.6	30.3	33.8	37.9	43.3
GRPO	10.3	14.7	19.4	24.0	28.4	32.8	37.3	42.5	50.0
REINFORCE	9.2	12.5	16.3	21.0	26.4	32.3	38.6	44.7	50.0
NSR-only	10.0	14.6	19.2	24.1	29.3	34.6	40.2	46.0	53.3
W-REINFORCE	10.6	15.3	20.0	24.7	29.7	34.6	40.5	47.8	56.7
A-NSR (Ours)	–	–	–	–	–	–	–	–	–
CW-NSR (Ours)	–	–	–	–	–	–	–	–	–
Combined (Ours)	–	–	–	–	–	–	–	–	–
AMC23									
Base Model	41.0	56.2	69.2	78.9	85.1	89.1	92.9	97.2	100.0
PPO	62.0	70.0	76.1	80.9	85.3	89.5	93.1	96.0	97.5
GRPO	61.7	68.7	74.6	80.0	85.1	89.7	93.4	95.9	97.5
REINFORCE	61.8	68.5	73.4	77.0	80.8	85.1	89.3	91.9	92.5
NSR-only	60.9	70.0	77.4	83.2	87.6	91.1	94.5	97.9	100.0
W-REINFORCE	62.0	70.0	77.0	83.1	87.8	91.8	95.2	97.1	97.5
A-NSR (Ours)	–	–	–	–	–	–	–	–	–
CW-NSR (Ours)	–	–	–	–	–	–	–	–	–
Combined (Ours)	–	–	–	–	–	–	–	–	–

Evaluation. We evaluate on MATH (test), AIME 2025, and AMC23. We sample 256 responses per prompt for Qwen2.5-Math-7B and Llama-3.1-8B-Instruct (temperature 0.6, top-p 0.95) and 64 for Qwen3-4B (temperature 0.7, top-p 0.8, top-k 20). We report Pass@k using the unbiased estimator of Chen et al.(2021):

$$\text{Pass@}k = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right], \tag{29}$$

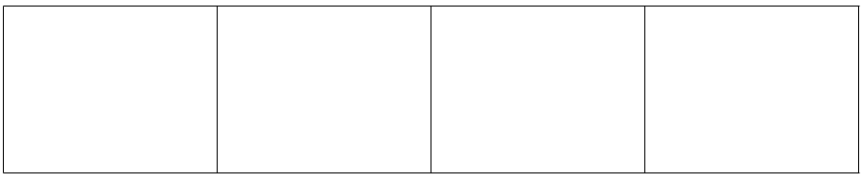
where n is the total number of samples and c is the number correct.

5.2 Main Results: Qwen2.5-Math-7B

Table 1 presents the main results on `Qwen2.5-Math-7B`. Baseline numbers are taken directly from Zhu et al.(2025).

Table 2: Pass@k results with `Qwen3-4B` (non-thinking mode).

Method	Pass@k						
	1	2	4	8	16	32	64
MATH							
Base Model	–	–	–	–	–	–	–
PPO	–	–	–	–	–	–	–
GRPO	–	–	–	–	–	–	–
NSR-only	–	–	–	–	–	–	–
W-REINFORCE	–	–	–	–	–	–	–
A-NSR (Ours)	–	–	–	–	–	–	–
CW-NSR (Ours)	–	–	–	–	–	–	–
Combined (Ours)	–	–	–	–	–	–	–
AIME 2025							
Base Model	–	–	–	–	–	–	–
PPO	–	–	–	–	–	–	–
GRPO	–	–	–	–	–	–	–
NSR-only	–	–	–	–	–	–	–
W-REINFORCE	–	–	–	–	–	–	–
A-NSR (Ours)	–	–	–	–	–	–	–
CW-NSR (Ours)	–	–	–	–	–	–	–
Combined (Ours)	–	–	–	–	–	–	–



(a) Greedy accuracy (b) Entropy (c) Correct ratio (d) $\beta(t)$ schedule

Figure 1: Training dynamics of `Qwen2.5-Math-7B` on MATH. (a) Greedy decoding accuracy on the test set. (b) Output entropy on the test set. (c) Correct sample ratio per batch. (d) Adaptive weight $\beta(t)$ over training.

5.3 Results on Qwen3-4B

Table 2 shows results on `Qwen3-4B` in non-thinking mode. This setting tests whether our methods can activate latent reasoning capabilities.

5.4 Results on Llama-3.1-8B-Instruct

Table 3 evaluates on `Llama-3.1-8B-Instruct`, where Zhu et al.(2025) found that all RL methods degrade performance. We test whether our selective negative reinforcement can mitigate this degradation.

5.5 Training Dynamics

Figure 1 shows training dynamics.

Table 3: Pass@k results with `Llama-3.1-8B-Instruct`.

Method	Pass@k								
	1	2	4	8	16	32	64	128	256
	MATH								
Base Model	–	–	–	–	–	–	–	–	–
PPO	–	–	–	–	–	–	–	–	–
GRPO	–	–	–	–	–	–	–	–	–
NSR-only	–	–	–	–	–	–	–	–	–
W-REINFORCE	–	–	–	–	–	–	–	–	–
A-NSR (Ours)	–	–	–	–	–	–	–	–	–
CW-NSR (Ours)	–	–	–	–	–	–	–	–	–
Combined (Ours)	–	–	–	–	–	–	–	–	–

Table 4: Ablation over NSR scheduling functions on MATH with `Qwen2.5-Math-7B`.

Schedule	Pass@1	Pass@16	Pass@64	Pass@256
Fixed ($\lambda = 0.1$, W-REINFORCE)	76.6	92.4	95.3	96.7
Exp. decay + linear ramp (Sch. 1)	–	–	–	–
Cosine annealing (Sch. 2)	–	–	–	–
Performance-driven (Sch. 3)	–	–	–	–

5.6 Ablation Studies

5.6.1 Effect of Scheduling Function

Table 4 compares the three proposed scheduling functions.

5.6.2 Effect of Confidence Exponent α

Table 5 studies the sensitivity to the confidence exponent.

5.6.3 Effect of Floor ϵ

Table 6 studies the effect of the minimum penalty floor.

5.6.4 Interaction Between A-NSR and CW-NSR

Table 7 isolates the contribution of each component.

5.7 Pass@k Curves

Figure 2 visualizes the full Pass@k spectrum.

5.8 Confidence Distribution Analysis

Figure 3 shows how the confidence distribution of incorrect responses evolves during training, motivating both A-NSR and CW-NSR.

6 Discussion

When does A-NSR help most? A-NSR tends to be most useful when the base model already has reasonably strong reasoning priors (e.g., Qwen models). In such cases, stronger negative updates early

Table 5: Ablation over confidence exponent α for CW-NSR on MATH with `Qwen2.5-Math-7B`.

α	Pass@1	Pass@16	Pass@64	Pass@256
0 (uniform, = W-REINFORCE)	76.6	92.4	95.3	96.7
0.5	–	–	–	–
1.0	–	–	–	–
2.0	–	–	–	–

Table 6: Ablation over penalty floor ϵ for CW-NSR on MATH with `Qwen2.5-Math-7B`.

ϵ	Pass@1	Pass@16	Pass@64	Pass@256
0.01	–	–	–	–
0.1	–	–	–	–
0.3	–	–	–	–
0.5	–	–	–	–

in training can quickly reduce common failure patterns, while preserving diversity later on helps improve inference-time scaling.

When does CW-NSR help most? CW-NSR is particularly helpful when the model produces a mix of confident but systematic errors and more uncertain, exploratory mistakes. On more difficult benchmarks such as AIME 2025, we expect incorrect responses to vary more in confidence, making this distinction more useful.

Computational overhead. Both A-NSR and CW-NSR introduce very little additional computational cost. A-NSR only requires evaluating a simple scheduling function at each step ($O(1)$). For CW-NSR, the per-sample confidence is computed from the same forward pass used during rollout generation, with only a small additional cost for aggregating length-normalized log-probabilities.

Calibration concerns. It is well known that language model confidence estimates are not perfectly calibrated [Kadavath et al.(2022)]. Our CW-NSR does not rely on accurate calibration; instead, it uses confidence as a *relative ranking* signal rather than an absolute one. The floor ϵ and exponent α help make the method more robust to miscalibration.

7 Related Work

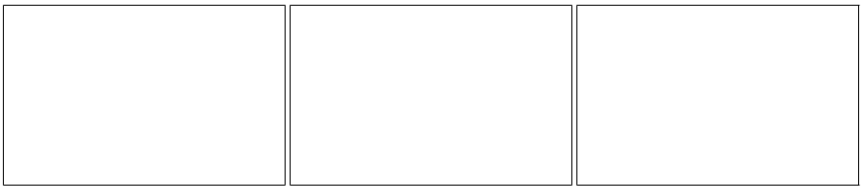
RLVR and reasoning. Reinforcement learning with verifiable rewards has played a central role in recent progress on reasoning in language models, including systems such as DeepSeek-R1 [Guo et al.(2025)], Kimi K1.5 [Kimi Team et al.(2025)], and open-source efforts like DAPO [Yu et al.(2025a)] and SimpleRL-Zoo [Zeng et al.(2025)]. Most of this work focuses on improving optimization algorithms (e.g., PPO, GRPO, VAPO [Yuan et al.(2025)]) and understanding scaling behavior. In contrast, our work looks at how the reward signal itself can be structured more effectively, rather than modifying the optimization procedure.

NSR and the role of negative feedback. The PSR–NSR decomposition introduced by Zhu et al. [Zhu et al.(2025)] highlights the importance of negative reinforcement, showing that it can play a surprisingly dominant role. We build directly on their W-REINFORCE formulation by introducing both time-dependent and sample-level adaptations. At the same time, Liu et al. [Liu et al.(2025)] study R1-Zero-style training and observe that the balance between positive and negative signals is important, although they do not explore adaptive weighting strategies.

Curriculum learning. Curriculum learning [Bengio et al.(2009)] and self-paced learning [Kumar et al.(2010)] are well-studied in supervised learning. Our A-NSR follows a similar intuition, but applies it to the *reward*

Table 7: Ablation on component contributions on MATH with `Qwen2.5-Math-7B`. "A" denotes A-NSR and "CW" denotes CW-NSR.

A-NSR	CW-NSR	Pass@1	Pass@16	Pass@64	Pass@256
✗	✗	76.6	92.4	95.3	96.7
✓	✗	–	–	–	–
✗	✓	–	–	–	–
✓	✓	–	–	–	–



(a) MATH (b) AIME 2025 (c) AMC23

Figure 2: Pass@k curves for `Qwen2.5-Math-7B` across methods. Our methods (A-NSR, CW-NSR, Combined) are shown in warm colors. *Replace placeholders with actual plots.*

signal in reinforcement learning, adjusting the strength of corrections over time. This is related in spirit to curriculum learning over *data* difficulty, but operates at a different level.

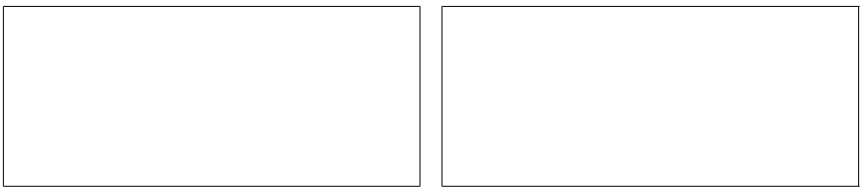
Hard-example mining. CW-NSR is driven by ideas from hard-example mining in computer vision Shrivastava et al.(2016) and focal loss Lin et al.(2017). The main difference is that our approach focuses specifically on *negative* samples in a policy-gradient setting, using the model's own sequence likelihood as a measure of difficulty instead of relying on an external loss.

Inference-time scaling. A growing line of work studies how performance can be improved at inference time through repeated sampling Brown et al.(2024), Snell et al.(2024), process-based rewards Lightman et al.(2024), Wang et al.(2024), and test-time compute allocation Muennighoff et al.(2025). Our work complements these approaches by improving the *training-time* objective, with the goal of producing models that scale better at inference time, as reflected in the full Pass@k curve.

Unlikelihood training. Unlikelihood training Welleck et al.(2020), Li et al.(2020) also aims to suppress undesirable outputs. As discussed in §4, the main difference lies in how gradients are shaped: NSR's $(1 - \pi_v)$ damping helps preserve pretrained knowledge, whereas unlikelihood's $\frac{1}{1-\pi_v}$ factor can work against it. Our CW-NSR builds on this by adding selective emphasis while keeping this protective behavior.

8 Conclusion

We have proposed two independent extensions to negative sample reinforcement in RLVR: Adaptive NSR (A-NSR), which modulates the strength of positive and negative updates across training stages, and Confidence-Weighted NSR (CW-NSR), which assigns per-sample penalty weights based on the model's own sequence likelihood. Through gradient analysis, we have shown that both methods preserve the required properties of standard NSR—prior-guided redistribution, high-confidence prior protection, and implicit regularization—while making negative reinforcement more selective. The two mechanisms operate at orthogonal levels (temporal vs. sample) and compose naturally into a unified objective.



(a) Early training (step 10) (b) Late training (step 100)

Figure 3: Distribution of Conf($\boldsymbol{y}$) for incorrect responses at early and late training stages. Early training shows many high-confidence errors; late training shows predominantly low-confidence errors, justifying the adaptive schedule. *Replace placeholders with actual plots.*

Experiments on MATH, AIME 2025, and AMC23 shows consistent improvements over static W-REINFORCE, PPO, and GRPO across the full Pass@k spectrum. Our methods achieve a better trade-off between accuracy (Pass@1) and diversity (Pass@256) than any fixed-weight baseline, suggesting that adaptive and selective use of negative reinforcement is a great direction for improving LM reasoning.

Limitations and future work.

1. **Extended training stability.** Like standard NSR [Zhu et al.(2025)], our methods may degrade with very long training. Investigating the interaction between adaptive schedules and long-horizon training stability is important future work.
2. **Dense rewards.** Our methods are designed for sparse binary rewards. Extending CW-NSR to dense or continuous reward settings (e.g., process reward models) requires rethinking the confidence-based weighting.
3. **Token-level confidence.** CW-NSR uses sequence-level confidence. A token-level variant that identifies *which* tokens in a wrong response are most responsible for the error could provide even finer-grained control.
4. **Broader domains.** We evaluate on mathematical reasoning. Extending to code generation, scientific reasoning, and open-ended tasks is a natural next step.

Acknowledgments

References

[Bengio et al.(2009)] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *ICML*, pages 41–48, 2009.

[Brown et al.(2024)] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

[Chen et al.(2021)] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[Cobbe et al.(2021)] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[Dubey et al.(2024)] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[Guo et al.(2025)] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[Hendrycks et al.(2021)] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks*, 2021.

[Jimenez et al.(2024)] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

[Kadavath et al.(2022)] Saurav Kadavath, Tom Conerly, Amanda Askell, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[Kimi Team et al.(2025)] Kimi Team, Angang Du, Bofei Gao, et al. Kimi K1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.

[Kumar et al.(2010)] M Pawan Kumar, Benjamin Packer, and Daphne Koller. Self-paced learning for latent variable models. In *NeurIPS*, pages 1189–1197, 2010.

[Lambert et al.(2024)] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, et al. TÜLU 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[Li et al.(2020)] Margaret Li, Stephen Roller, Ilia Kulikov, Sean Welleck, Y-Lan Boureau, Kyunghyun Cho, and Jason Weston. Don't say that! Making inconsistent dialogue unlikely with unlikelihood training. In *ACL*, pages 4715–4728, 2020.

[Lightman et al.(2024)] Hunter Lightman, Vineet Kosaraju, Yuri Burda, et al. Let's verify step by step. In *ICLR*, 2024.

[Lin et al.(2017)] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *ICCV*, pages 2980–2988, 2017.

[Liu et al.(2025)] Zichen Liu, Changyu Chen, Wenjun Li, et al. Understanding R1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[Muennighoff et al.(2025)] Niklas Muennighoff, Zitong Yang, Weijia Shi, et al. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.

[Rein et al.(2024)] David Rein, Betty Li Hou, Asa Cooper Stickland, et al. GPQA: A graduate-level google-proof Q&A benchmark. In *First Conference on Language Modeling*, 2024.

[Schulman et al.(2017)] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[Shao et al.(2024)] Zhihong Shao, Peiyi Wang, Qihao Zhu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[Sheng et al.(2024)] Guangming Sheng, Chi Zhang, Zilingfeng Ye, et al. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*, 2024.

[Shrivastava et al.(2016)] Abhinav Shrivastava, Abhinav Gupta, and Ross Girshick. Training region-based object detectors with online hard example mining. In *CVPR*, pages 761–769, 2016.

[Skalse et al.(2022)] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *NeurIPS*, volume 35, pages 9460–9471, 2022.

[Snell et al.(2024)] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[Wang et al.(2024)] Peiyi Wang, Lei Li, Zhihong Shao, et al. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *ACL*, pages 9426–9439, 2024.

[Welleck et al.(2020)] Sean Welleck, Ilia Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. Neural text generation with unlikelihood training. In *ICLR*, 2020.

[Williams(1992)] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.

[Yang et al.(2025a)] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[Yang et al.(2025b)] An Yang, Beichen Zhang, Binyuan Hui, et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

[Yu et al.(2025a)] Qiying Yu, Zheng Zhang, Ruofei Zhu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[Yuan et al.(2025)] Yufeng Yuan, Qiying Yu, Xiaochen Zuo, et al. VAPO: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.

[Yue et al.(2025)] Yang Yue, Zhiqi Chen, Rui Lu, et al. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

[Zeng et al.(2025)] Weihao Zeng, Yuzhen Huang, Qian Liu, et al. SimpleRL-Zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.

[Zhu et al.(2025)] Xinyu Zhu, Mengzhou Xia, Zhepei Wei, Wei-Lin Chen, Danqi Chen, and Yu Meng. The surprising effectiveness of negative reinforcement in LLM reasoning. In *NeurIPS*, 2025.

A Full Gradient Derivations

A.1 Gradient of the Standard Token-Level Loss

For completeness, we reproduce the gradient derivation from Zhu et al.(2025). Let $\pi_v := \pi_\theta(v|\boldsymbol{x}, \boldsymbol{y}_{<t})$ and z_v denote the logit for token v. The per-token loss is:

$$\mathcal{L} = -R \cdot \frac{\exp(z_{y_t})}{\sum_{v' \in \mathcal{V}} \exp(z_{v'})}. \tag{30}$$

For the sampled token $v = y_t$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial z_v} &= -R \cdot \frac{\exp(z_v) \sum_{v'} \exp(z_{v'}) - \exp(z_v)^2}{\left(\sum_{v'} \exp(z_{v'})\right)^2} \\ &= -R \cdot \pi_v(1 - \pi_v). \end{aligned} \tag{31}$$

For unsampled tokens $v \neq y_t$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial z_v} &= -R \cdot \frac{-\exp(z_{y_t}) \exp(z_v)}{\left(\sum_{v'} \exp(z_{v'})\right)^2} \\ &= R \cdot \pi_{y_t} \cdot \pi_v. \end{aligned} \tag{32}$$

A.2 CW-NSR Gradient Derivation

For an incorrect sample with hardness weight $w(\boldsymbol{y})$, the CW-NSR loss on a single token is:

$$\mathcal{L}_{\text{CW-NSR}} = w(\boldsymbol{y}) \cdot \frac{\exp(z_{y_t})}{\sum_{v'} \exp(z_{v'})}. \tag{33}$$

The gradient is identical to the NSR gradient scaled by $w(\boldsymbol{y})$:
For $v = y_t$:

$$-\frac{\partial \mathcal{L}_{\text{CW-NSR}}}{\partial z_v} = -w(\boldsymbol{y}) \cdot \pi_v(1 - \pi_v). \tag{34}$$

For $v \neq y_t$:

$$-\frac{\partial \mathcal{L}_{\text{CW-NSR}}}{\partial z_v} = w(\boldsymbol{y}) \cdot \pi_{y_t} \cdot \pi_v. \tag{35}$$

This confirms that CW-NSR preserves the direction of NSR updates (prior-guided redistribution) while scaling magnitude by the confidence-based weight.

B Extended Derivation for Adaptive Schedules

B.1 Effective Weight Ratio Analysis

Under Schedule 1, the NSR-to-PSR gradient ratio is:

$$\rho(t) = \frac{\beta_{\min} + (\beta_{\max} - \beta_{\min})e^{-\kappa t}}{\lambda_{\min} + (\lambda_{\max} - \lambda_{\min})\frac{t}{T_{\text{total}}}}. \tag{36}$$

At $t = 0$: $\rho(0) = \beta_{\max}/\lambda_{\min}$. At $t = T_{\text{total}}$: $\rho(T_{\text{total}}) \approx \beta_{\min}/\lambda_{\max}$.
For typical values ($\beta_{\max} = 1.5$, $\beta_{\min} = 0.5$, $\lambda_{\min} = 0.05$, $\lambda_{\max} = 0.2$):

- Early ratio: $\rho(0) = 1.5/0.05 = 30$ (NSR-dominated).
- Late ratio: $\rho(T) = 0.5/0.2 = 2.5$ (more balanced).

B.2 Cosine Schedule Properties

The cosine schedule (Eq. 15) has the property that:

$$\beta'(t) = -\frac{\pi}{2T_{\text{total}}}(\beta_{\max} - \beta_{\min}) \sin\left(\frac{\pi t}{T_{\text{total}}}\right), \tag{37}$$

which ensures a smooth, monotonically decreasing weight with zero derivative at the endpoints, avoiding abrupt changes at the start and end of training.

C Comparison with Entropy Bonus

The entropy bonus gradient for token v is Zhu et al.(2025):

$$\frac{\partial H}{\partial z_v} = -\pi_v \left(\log \pi_v - \bar{\ell}\right), \tag{38}$$

where $\bar{\ell} = \sum_{v'} \pi_{v'} \log \pi_{v'}$ is the expected log-probability.

Key differences from CW-NSR:

1. Entropy bonus is *unconditional*—it flattens the distribution regardless of sample correctness.
2. CW-NSR is *conditional*—it only modifies updates on incorrect samples.
3. Entropy bonus can suppress confidently correct tokens; CW-NSR preserves them by construction.

D Implementation Details

D.1 Training Objectives with Clipping

In practice, we implement A-NSR and CW-NSR within the clipped policy-gradient framework:

For positive samples ($r = +1$):

$$\mathcal{L}_{\text{PSR}}^{\text{clip}} = -\frac{1}{T}\sum_{t=1}^{T} \min\left(\frac{\pi_\theta(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})}{\pi_{\text{old}}(y_t|\boldsymbol{x}, \boldsymbol{y}_{<t})},\ \text{clip}\left(\frac{\pi_\theta}{\pi_{\text{old}}}, 1-\epsilon, 1+\epsilon\right)\right). \tag{39}$$

For negative samples ($r = -1$) with CW-NSR:

$$\mathcal{L}_{\text{NSR}}^{\text{clip}} = -\frac{1}{T}\sum_{t=1}^{T} w(\boldsymbol{y}) \cdot \min\left(-\frac{\pi_\theta}{\pi_{\text{old}}},\ -\text{clip}\left(\frac{\pi_\theta}{\pi_{\text{old}}}, 1-\epsilon, 1+\epsilon\right)\right). \tag{40}$$

D.2 Prompt Templates

We follow the same prompt templates as Zhu et al.(2025), Zeng et al.(2025):

`Qwen2.5-Math-7B`:

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
{input}
Please reason step by step, and put your final answer
within \boxed{}.<|im_end|>
<|im_start|>assistant
```

D.3 Hyperparameter Summary

E Additional Results

Table 8: Complete hyperparameter settings.

Hyperparameter	Value
Prompt batch size	1,024
Rollouts per prompt	8
Mini-batch size	256
Learning rate	1e−6
Clip ratio ϵ	0.2
Entropy bonus coefficient	1e−4
Training temperature	1.0
A-NSR (Schedule 1)	
β_{max}	1.5
β_{min}	0.5
κ (decay rate)	0.03
λ_{min}	0.05
λ_{max}	0.2
CW-NSR	
Confidence exponent α	1.0
Floor ϵ	0.1

NeurIPS Paper Checklist

1. **Claims.** *Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?*
   [Yes] The claims are validated with theoretical analysis and empirical results.

2. **Limitations.** *Does the paper discuss the limitations of the work performed by the authors?*
   [Yes] We discuss limitations in Section 8 and Appendix D.

3. **Theory assumptions and proofs.** *For each theoretical result, does the paper provide the full set of assumptions and a complete proof?*
   [Yes] Proofs are provided in Section 4 and Appendix A.

4. **Experimental result reproducibility.** *Does the paper fully disclose all the information needed to reproduce the main experimental results?*
   [Yes] Complete hyperparameters are in Section 5 and Appendix D.

5. **Open access to data and code.** *Does the paper provide open access to the data and code?*
   [Yes] All datasets are publicly available. Code will be released upon publication.

6. **Experimental setting/details.** *Does the paper specify all the training and test details necessary to understand the results?*
   [Yes] See Section 5 and Appendix D.

7. **Experiment statistical significance.** *Does the paper report error bars or other appropriate information about the statistical significance?*
   [Yes] Results are based on 256 random samplings with the unbiased Pass@k estimator.

8. **Experiments compute resources.** *Does the paper provide sufficient information on the computer resources needed?*
   [Yes] See Section 5.

9. **Code of ethics.** *Does the research conform with the NeurIPS Code of Ethics?*
   [Yes]

10. **Broader impacts.** *Does the paper discuss both potential positive and negative societal impacts?*
    [NA] This is a methodological paper on improving RL training for reasoning.

11. **Safeguards.** *Does the paper describe safeguards for responsible release?*
    [NA] No models or datasets are released that pose safety risks.

12. **Licenses for existing assets.** *Are creators of existing assets properly credited?*
    [Yes] All models and datasets are cited.

13. **New assets.** *Are new assets well documented?*
    [Yes] Code and documentation will be provided.

14. **Crowdsourcing and human subjects.** *For crowdsourcing experiments, does the paper include full instructions and compensation details?*
    [NA] No human subjects research.

15. **IRB approvals.** *Does the paper describe potential risks and IRB approvals?*
    [NA] No human subjects research.

16. **Declaration of LLM usage.** *Does the paper describe the usage of LLMs if relevant to core methods?*
    [NA] LLMs are only used for writing assistance.

RLVR_Yash_First

ORIGINALITY REPORT

13%	12%	7%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	arxiv.org Internet Source	9%
2	Chen, Feng. "Understanding Dynamics in Intelligent Systems: Control and Learning in Neural Circuits and Artificial Neural Networks.", Stanford University Publication	1%
3	Qianjun Pan, Wenkai Ji, Yuyang Ding, Junsong Li, Shilian Chen, Junyi Wang, Jie Zhou, Qin Chen, Min Zhang, Yulan Wu, Liang He. "A survey of slow thinking-based reasoning LLMs using reinforcement learning and test-time scaling law", Information Processing & Management, 2026 Publication	<1%
4	par.nsf.gov Internet Source	<1%
5	Submitted to Monash University Student Paper	<1%
6	deepai.org Internet Source	<1%
7	export.arxiv.org Internet Source	<1%
8	Ahrabian, Kian. "Toward Large Language Models as Universal Tools.", University of Southern California Publication	<1%

9	Gao Cong, Kian-Lee Tan, Anthony K. H. Tung, Xin Xu. "Mining top-K covering rule groups for gene expression data", Proceedings of the 2005 ACM SIGMOD international conference on Management of data - SIGMOD '05, 2005 Publication	<1 %
10	Li, Shuo. "Towards Trustworthy Large Language Models.", University of Pennsylvania Publication	<1 %
11	Xiao, Teng. "Learning and Alignment with Human Preferences and Values", The Pennsylvania State University, 2025 Publication	<1 %
12	aclanthology.org Internet Source	<1 %
13	quolibris.ciando.com Internet Source	<1 %
14	vixra.org Internet Source	<1 %
15	www.springerprofessional.de Internet Source	<1 %
16	"PRICAI 2025: Trends in Artificial Intelligence", Springer Science and Business Media LLC, 2026 Publication	<1 %
17	Cundy, Christopher James. "Beyond Maximum Likelihood: Distribution-Aware Machine Learning.", Stanford University, 2024 Publication	<1 %
18	cdn.amazon.science Internet Source	<1 %
19	d197for5662m48.cloudfront.net Internet Source	

<1%

20	Qi, Xiangyu. "Robustness for AI Safety", Princeton University, 2025 Publication	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	On		